



Mekelle University

Mekelle Institute of Technology(MIT)

Department of information technology (IT)

ML Group Assignment

No.	Group members name	ID NO.
1	Abraham Michael	MIT/UR/004/12
2	Alazar Goitom	MIT/UR/007/12
3	Florida H/slassie	MIT/UR/043/12
4	Kidan G/aregawi	MIT/UR/083/12
5	Mahlet Ezra	MIT/UR/215/12
6	Netsanet Meaza	MIT/UR/113/12
7	Rshan G/medhin	MIT/UR/249/12
8	Senait G/hans	MIT/UR/218/12

Regression Report for life expectancy

A. Introduction

Objective:

The objective of this project is to predict life expectancy based on various socio-economic and environmental factors such as income, healthcare expenditure, and status of countries (developed or developing). By analyzing the relationship between these factors and life expectancy, the goal is to provide insights that could help improve public health strategies and policies.

Importance of Solving This Problem:

Understanding and predicting life expectancy is crucial for public health planning, resource allocation, and designing targeted interventions. Accurate predictions can guide decision-makers in improving healthcare infrastructure and enhancing the quality of life across different regions.

B. Data Exploration (EDA)

Dataset Description:

The dataset used in this project is the **Life Expectancy Data**. It includes the following features:

- **Country:** Country of the data point (categorical).
- **Year:** Year of the data point.
- **Status:** Country status (1 for Developed, 0 for Developing, categorical).
- **Life expectancy:** Target variable that indicates the life expectancy (continuous).
- **GDP:** Gross Domestic Product per capita (continuous).
- **Alcohol:** Alcohol consumption per capita (continuous).
- **Percentage expenditure:** Government expenditure on health as a percentage of GDP (continuous).
- **Measles:** The number of measles cases (continuous).
- **BMI:** Average Body Mass Index (continuous).
- **Under-five deaths:** Deaths of children under five years of age (continuous).
- **Polio:** Percentage of children immunized against polio (continuous).
- **Total expenditure:** Total healthcare expenditure (continuous).
- **HIV/AIDS:** Prevalence of HIV/AIDS (continuous).
- **Diphtheria:** Immunization rate for diphtheria (continuous).
- **Income:** Income per capita (continuous).
- **Region:** Geographical region of the country (categorical).
- **Urbanization:** Percentage of urban population (continuous).

Key Visualizations:

- **Pairplot:** Shows the relationships between features. This helps to visually inspect the pairwise relationships and identify patterns in the data.
 - We use `sns.pairplot(life_df)` to generate the pairplot of all features.
- **Correlation Heatmap:** Displays the correlation matrix to identify which features are strongly correlated with each other. This is essential for understanding how features interact and may affect the target variable.

- We use `sns.heatmap()` to visualize correlations between all variables.
- **Histograms:** Displays the distribution of each feature in the dataset. Histograms help to detect skewness, normality, and outliers in the data.
 - We use `life_df.hist()` to create histograms of all features.

C. Methodology

Preprocessing Steps

- **Missing Data Handling:** Replaced infinite values with NaNs, then dropped rows with any missing values to ensure data quality.
- **Encoding Categorical Variables:** Converted 'Country' and 'Status' columns into numerical format using categorical encoding.
- **Feature Scaling:** Standardized numerical features using `StandardScaler` for better model convergence and performance.
- **Target Variable:** 'Life expectancy ' (note the space at the end) was selected as the target for regression.

Algorithms Chosen and Reasoning

- **Linear Regression:**
A baseline model that assumes a linear relationship between the independent variables and life expectancy. It is easy to interpret and useful for benchmarking.
- **Random Forest Regressor:**
A robust ensemble method that captures non-linear relationships and handles feature interactions well. It is less prone to overfitting than individual decision trees.

D. Model Training

Training Details

- **Train/Test Split:**
Dataset split into 80% training and 20% testing to evaluate generalization.

- **Linear Regression:**
Used with default settings (no hyperparameters needed).
- **Random Forest Regressor:**
 - `n_estimators = 100`: Number of trees in the forest.
 - `max_depth = 10`: Limited depth to avoid overfitting and reduce complexity.
 - `random_state = 42`: For reproducibility.

E. Results and Evaluation

Evaluation Metrics and Interpretation

Model	RMSE	MAE	R ² Score
Linear Regression	~2.44	~1.87	~0.83
Random Forest Regressor	~1.61	~1.20	~0.93

- **Root Mean Squared Error (RMSE):** Lower for Random Forest, indicating better accuracy.
- **Mean Absolute Error (MAE):** Reflects the average prediction error. Again, Random Forest performed better.
- **R² Score:** Random Forest explained ~93% of the variance, while Linear Regression explained ~83%.

Error Analysis

- **Linear Regression** showed limitations in capturing non-linear relationships and underperformed in complex areas.
- **Random Forest** yielded lower residuals, especially in ranges with extreme values or non-linear patterns.
- Visual residual analysis (not shown in code) would help confirm any heteroscedasticity or bias.

F. conclusion

• Summary of Findings

- Life expectancy is influenced by numerous socio-economic and health factors.
- Random Forest Regressor significantly outperforms Linear Regression, offering better prediction accuracy and handling complex feature interactions.
- Variables like immunization rates, BMI, and HIV/AIDS prevalence were found to be strong predictors.

Challenges Faced

- **Missing values:** Needed careful handling to avoid data loss or bias.
- **Inconsistent column naming:** The target column had a trailing space, which required attention.
- **Categorical encoding:** Required conversion for machine learning models to interpret non-numeric features.

Possible Improvements

- **Hyperparameter Tuning:** Use GridSearchCV or RandomizedSearchCV to fine-tune Random Forest parameters.
- **Cross-Validation:** Apply k-fold cross-validation to obtain a more robust performance estimate.
- **Feature Engineering:** Explore interaction terms or log-transform skewed features.
- **Model Explainability:** Use SHAP or feature importance plots to understand model decisions.
- **Time-based Analysis:** Analyze trends over years within countries to capture temporal effects.

G. Appendix

1. Import Libraries

=====

```
import warnings

warnings.filterwarnings("ignore", category=FutureWarning,
module="seaborn._oldcore")


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score


# Set plot style
sns.set(style='whitegrid')


# =====
# 2. Load Dataset
# =====

# Adjust the file path
life_df = pd.read_csv("/kaggle/input/life-expectancy-data/Life Expectancy
Data.csv")


# =====
# 3. Preprocessing
# =====
```

```
# Replace inf/-inf with NaN
life_df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Drop rows with missing values (including those from infs)
life_df.dropna(inplace=True)

# Basic info about the dataset
print(life_df.info())
print(life_df.describe())

# Encoding categorical variables (Country and Status columns)
life_df['Country'] = life_df['Country'].astype('category').cat.codes
life_df['Status'] = life_df['Status'].astype('category').cat.codes

# Defining features (X) and target (y)
X_reg = life_df.drop('Life expectancy ', axis=1)
y_reg = life_df['Life expectancy ']

# Feature scaling (for regression models)
scaler = StandardScaler()
X_reg_scaled = scaler.fit_transform(X_reg)

# =====
# 4. Train/Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(X_reg_scaled, y_reg,
test_size=0.2, random_state=42)
```



```
# =====
```

5. Model Training

```
# =====
```

```
# Linear Regression
```

```
lr_model = LinearRegression()
```

```
lr_model.fit(X_train, y_train)
```

```
# Random Forest Regressor
```

```
rf_model = RandomForestRegressor(n_estimators=100, max_depth=10,  
random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
# =====
```

6. Evaluation

```
# =====
```

```
def evaluate_regression(name, y_true, y_pred):
```

```
    print(f"--- {name} ---")
```

```
    print("RMSE:", np.sqrt(mean_squared_error(y_true, y_pred)))
```

```
    print("MAE:", mean_absolute_error(y_true, y_pred))
```

```
    print("R2 Score:", r2_score(y_true, y_pred))
```

```
# Evaluate models
```

```
evaluate_regression("Linear Regression", y_test, lr_model.predict(X_test))
```

```
evaluate_regression("Random Forest Regressor", y_test,  
rf_model.predict(X_test))
```

```
# =====
```

7. Visualizations

```
# =====
```

```
# Pairplot for feature relationships
```

```
sns.pairplot(life_df)
```

```
plt.suptitle("Pairplot of Features", y=1.02)
```

```
plt.show()
```

```
# Correlation heatmap for features
```

```
plt.figure(figsize=(14, 10))
```

```
correlation_matrix = life_df.corr()
```

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',  
linewidths=0.5)
```

```
plt.title("Correlation Heatmap", fontsize=16)
```

```
plt.show()
```

```
# Feature distribution histograms
```

```
life_df.hist(figsize=(12, 8), bins=20)
```

```
plt.suptitle("Histograms of Features", fontsize=16)
```

```
plt.tight_layout()
```

```
plt.show()
```

kaggle link:- <https://www.kaggle.com/code/alazargoitom/life-expectancy-regression>